Vi xử lý (Microprocessor)

GVGD: Vũ Thế Đảng

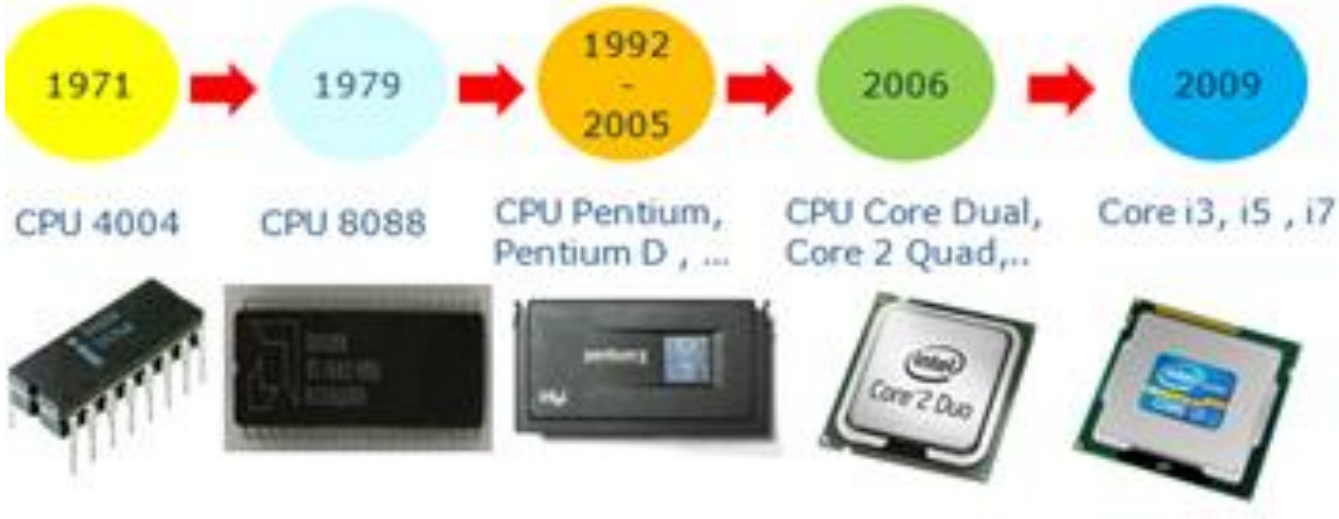
Email: dangvt@hcmute.edu.vn

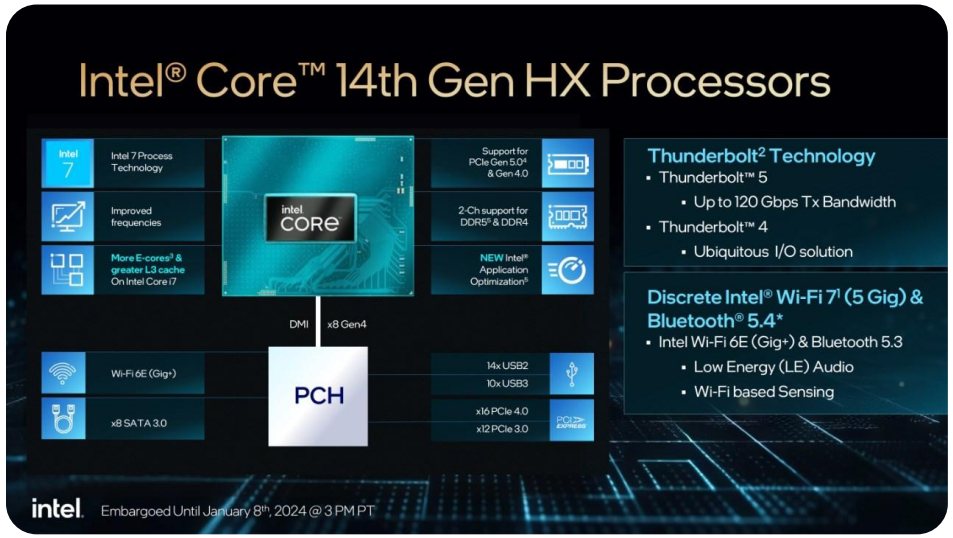
Khái niệm Vi xử lý vs Vi điều khiển

⚙️ Vi xử lý (Microprocessor)	🔧 Vi điều khiển (Microcontroller)
Vi xử lý là một bộ xử lý trung tâm (CPU) thuần túy, chuyên thực hiện các phép toán, xử lý dữ liệu và điều khiển hệ thống. Nó không tích hợp sẵn bộ nhớ hay các cổng I/O, nên cần thêm các linh kiện bên ngoài để hoạt động. **Đặc điểm:** • Chỉ bao gồm CPU. • Cần thêm RAM, ROM, bộ điều khiển I/O bên ngoài. • Thường dùng trong máy tính cá nhân, máy chủ, hệ thống xử lý phức tạp. • Có thể xử lý khối lượng dữ liệu lớn và đa nhiệm. Ví dụ: • Intel Core i7, AMD Ryzen, ARM Cortex-A.	Vi điều khiển là một hệ thống tích hợp trên một chip, bao gồm CPU, bộ nhớ, các cổng I/O, bộ định thời, ADC/DAC,... Tất cả đều nằm trong một mạch tích hợp duy nhất. **Đặc điểm:** • Tích hợp CPU + RAM + ROM + I/O + Timer + ADC/DAC. • Thiết kế nhỏ gọn, tiêu thụ điện năng thấp. • Thường dùng trong hệ thống nhúng, như máy giặt, lò vi sóng, robot, thiết bị IoT. • Dễ dàng giao tiếp với cảm biến, mô-đun không dây, thiết bị ngoại vi. Ví dụ: • ATmega328 (Arduino), STM32, PIC, ESP32.

So sánh theo các tiêu chí

Tiêu chí	Vi xử lý (VXL)	Vi điều khiển (VĐK)
Thành phần	Chỉ có CPU	CPU + RAM + ROM + I/O
Kích thước	Lớn	Nhỏ gọn
Ứng dụng	Máy tính, hệ thống lớn	Thiết bị nhúng, IoT
Chi phí	Cao	Thấp
Tiêu thụ điện năng	Cao	Thấp





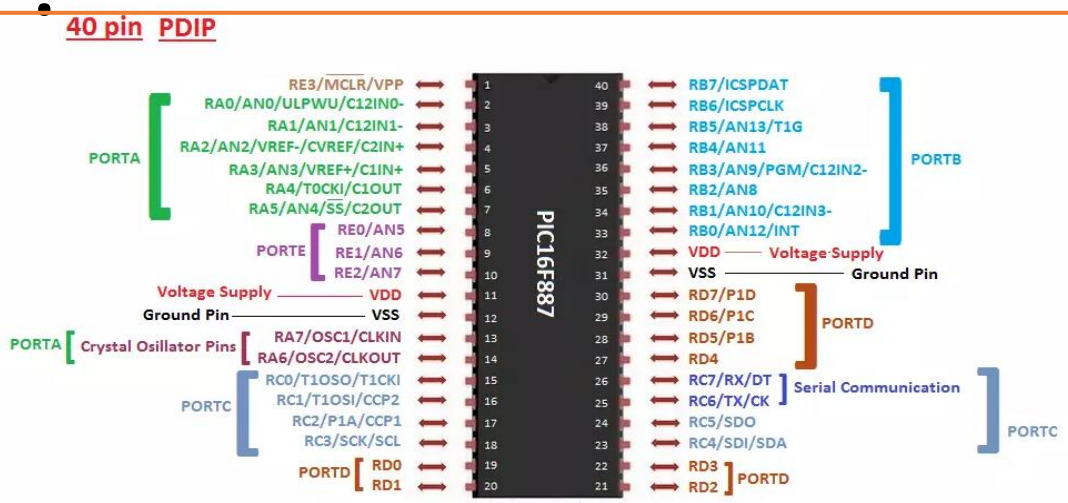
Chương 1: *ĐẶC TÍNH, CẤU TRÚC, CHỨC NĂNG CÁC PORT*

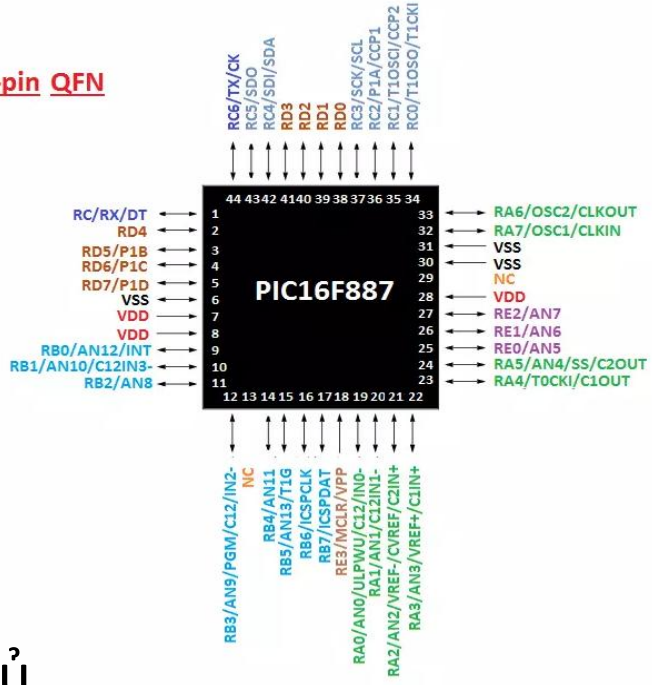
Vi điều khiển PIC16F887

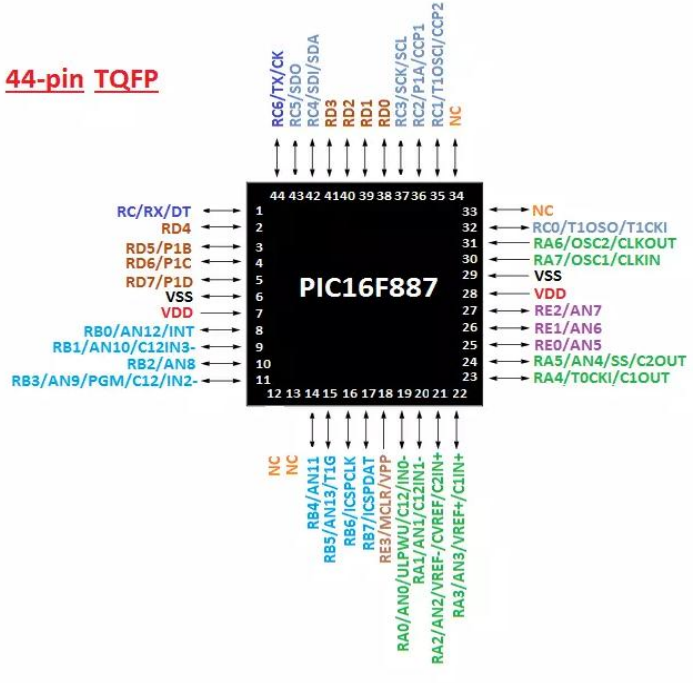
GVGD: Vũ Thế Đảng
Email: vuthedang@hcmute.edu.vn

1. Đặc tính kỹ thuật nổi bật của PIC16F887

- Vi điều khiển 8-bit, kiến trúc RISC (Reduced Instruction Set Computing)
- Tốc độ xử lý: lên đến 20 MHz
- Bộ nhớ:
  - Flash: 14 KB (8192 x 14 bit)
  - RAM: 368 bytes
  - EEPROM: 256 bytes
- Số chân: 40 chân (DIP) hoặc 44 chân (QFN/TQFP)
- Tích hợp bộ chuyển đổi ADC 10-bit với 14 kênh
- Có bộ định thời Timer0, Timer1, Timer2, Timer3
- Giao tiếp: UART, SPI, I2C
- Tích hợp bộ tạo xung clock nội
- Tiêu thụ điện năng thấp, hỗ trợ chế độ ngủ



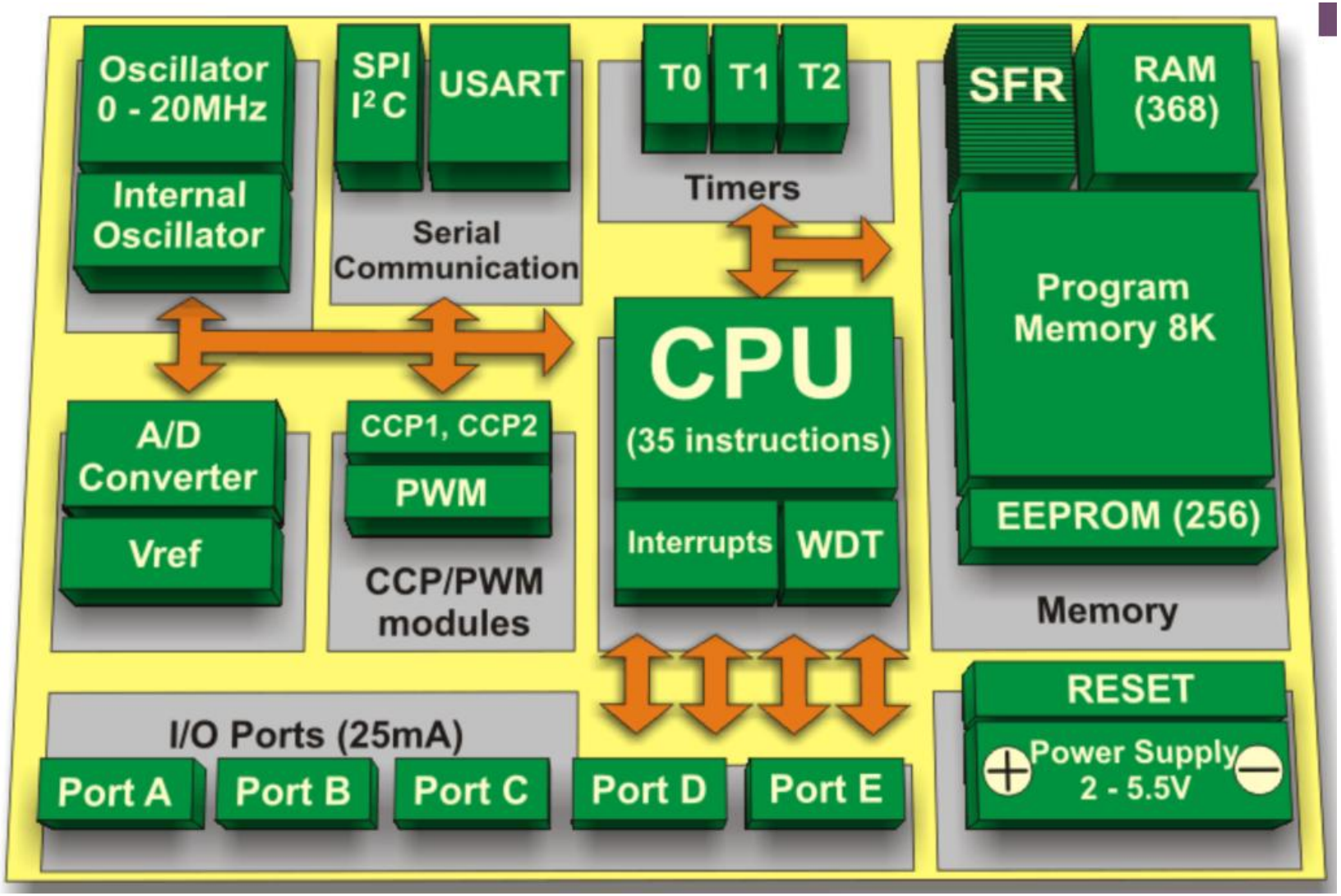




PIC16F887 Pinout

2. Cấu trúc tổng quát của PIC16F887

- **PIC16F887 gồm các khối chức năng chính:**
  - CPU: xử lý lệnh theo kiến trúc RISC
  - Bộ nhớ: gồm Flash, RAM, EEPROM
  - Các Port I/O: PORTA, PORTB, PORTC, PORTD, PORTE
  - Bộ chuyển đổi ADC: chuyển đổi tín hiệu analog sang digital
  - Timer/Counter: đo thời gian, tạo xung
  - Giao tiếp nối tiếp: UART, SPI, I2C
  - Ngắt (Interrupts): xử lý sự kiện bất thường
  - Watchdog Timer: giám sát hoạt động hệ thống



3. Chức năng và cấu trúc các Port

PIC16F887 có 5 Port I/O: A, B, C, D, E. Mỗi Port gồm 8 chân (trừ PORTE có 4 chân).

🟦 PORTA (RA0 – RA7)

- Chức năng: I/O số, đầu vào analog cho ADC
- Một số chân có chức năng đặc biệt như RA0 (AN0), RA1 (AN1), RA4 có thể dùng làm ngắt ngoài

🟧 PORTB (RB0 – RB7)

- Chức năng: I/O số, hỗ trợ ngắt ngoài tại RB0
- Có thể cấu hình từng chân là đầu vào hoặc đầu ra thông qua thanh ghi TRISB

🟥 PORTC (RC0 – RC7)

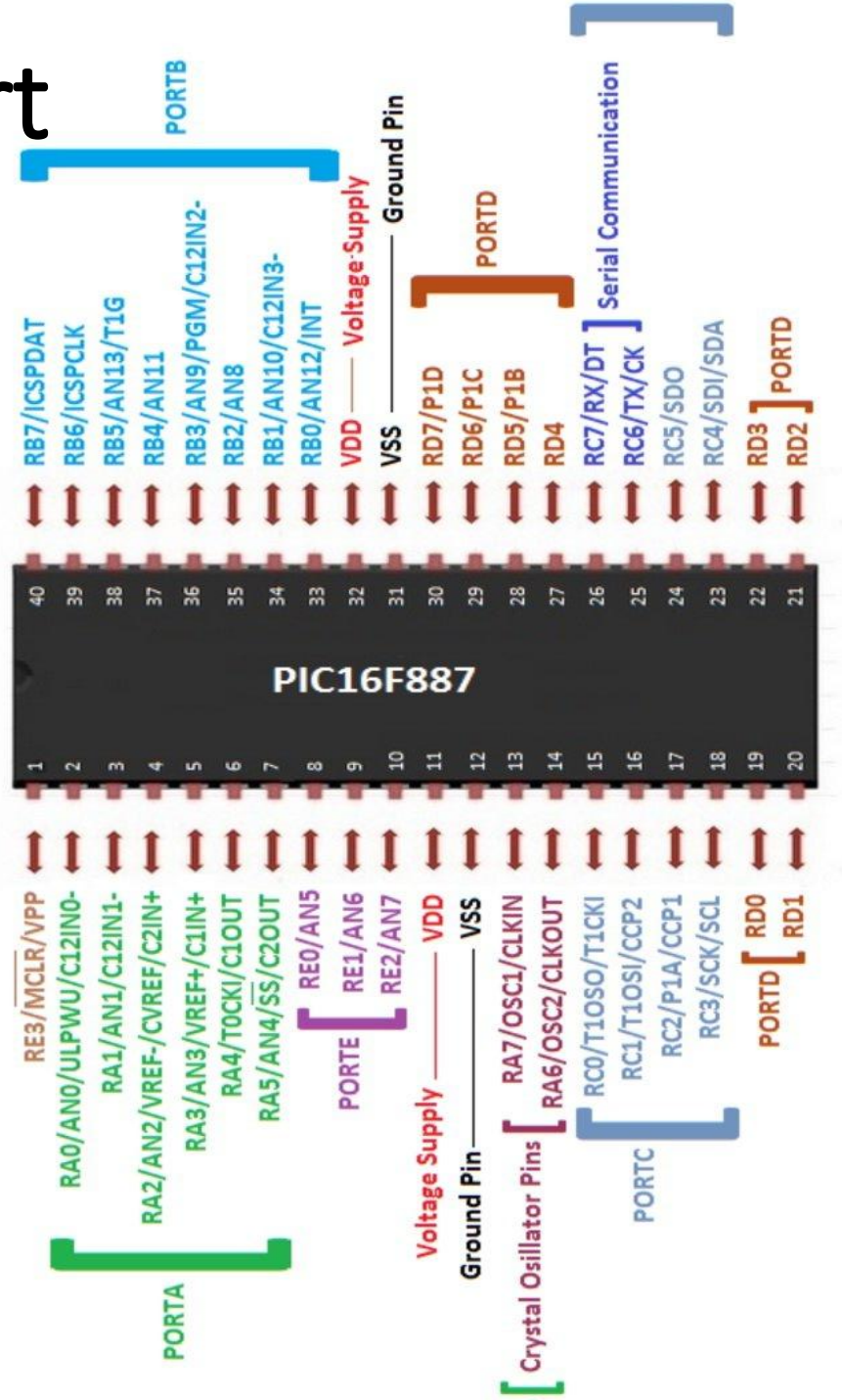
- Chức năng: I/O số, giao tiếp UART, SPI, I2C
- RC6 và RC7 thường dùng cho truyền nhận UART (TX/RX)

🟩 PORTD (RD0 – RD7)

- Chức năng: I/O số, thường dùng để giao tiếp với LCD hoặc thiết bị ngoại vi

🟧 PORTE (RE0 – RE3)

- Chức năng: đầu vào analog cho ADC, I/O số
- RE3 thường dùng làm chân Reset hoặc ngắt



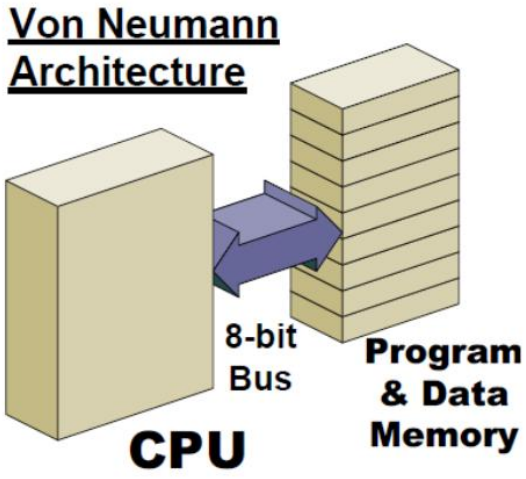
Chương II: *TỔ CHỨC BỘ NHỚ, THANH GHI*

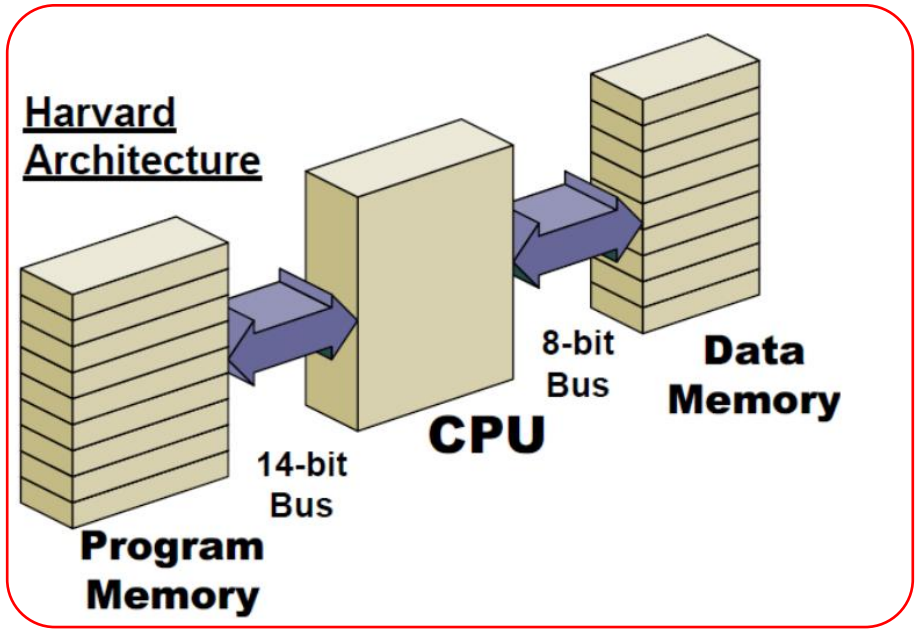
GVGD: Vũ Thế Đảng

Email: dangvt@hcmute.edu.vn

2.1. Kiến trúc bộ nhớ và thanh ghi của PIC16F887

2.1.1 I. Kiến trúc Bộ nhớ (Kiến trúc Harvard)





PIC16F887

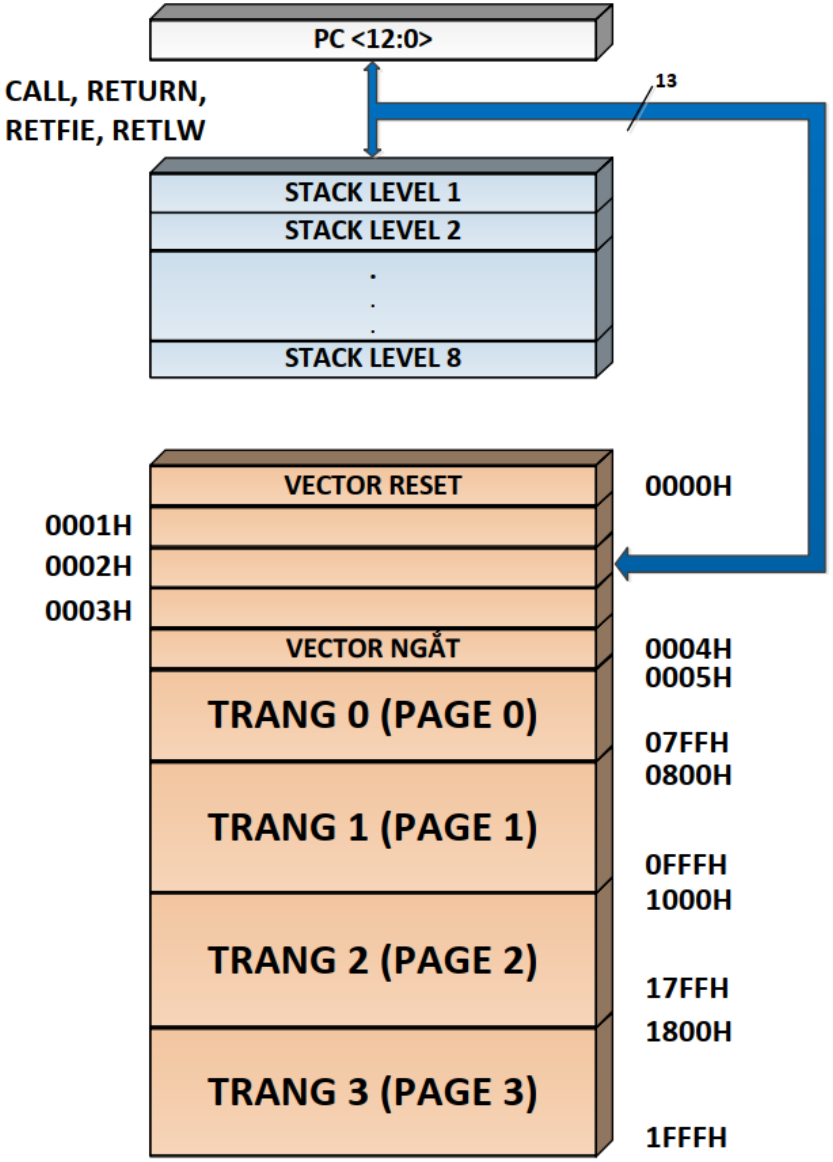
Vi điều khiển PIC16F887 sử dụng Kiến trúc Harvard.

- **Đặc điểm:** Trong kiến trúc Harvard, bộ nhớ được chia thành **hai loại độc lập**: bộ nhớ lưu chương trình và bộ nhớ lưu dữ liệu.
- **Ưu điểm:** Nhờ có hai bus độc lập để giao tiếp với hai loại bộ nhớ này, tốc độ truy xuất sẽ nhanh hơn

2.1.2. Tổ chức bộ nhớ chương trình của VĐK PIC16F887

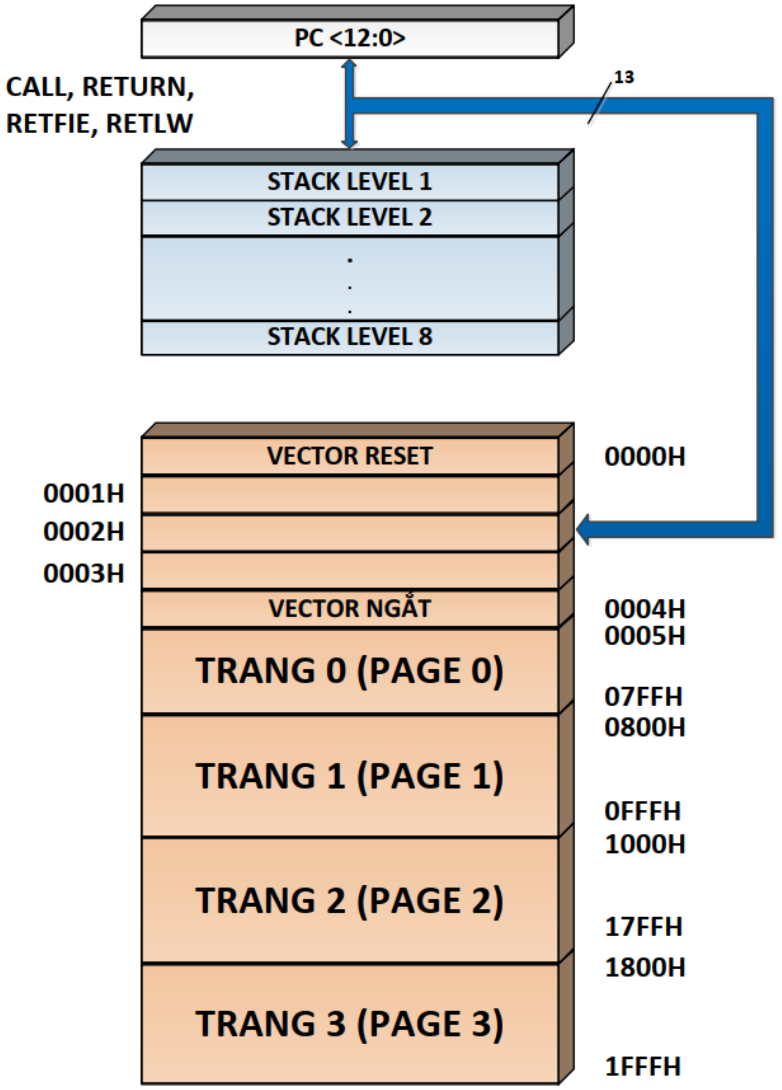
Bộ nhớ chương trình (Program Memory) của PIC16F887 là nơi lưu trữ chương trình sau khi đã được biên dịch.

- **Dung lượng:** 8K work.
- **Tổ chức:** Được chia thành **4 trang** (pages), mỗi trang có dung lượng 2K.
- **Đơn vị lưu trữ:** Mỗi ô nhớ chương trình lưu trữ **14 bit dữ liệu** (bao gồm cả mã lệnh và dữ liệu). ➔ giúp PIC tiết kiệm được một chu kỳ đọc dữ liệu so với các kiến trúc sử dụng đơn vị byte, ➔ tăng tốc độ thực hiện chương trình.
- **Thanh ghi quản lý:** Thanh ghi **Bộ đếm Chương trình (PC)** có độ dài **13 bits** quản lý 8192 ô nhớ (8K).
- **Vector địa chỉ:**
  - **Reset:** Khi PIC bị reset, thanh ghi PC có giá trị **0000H**.
  - **Ngắt (Interrupt):** Khi có ngắt tác động, PIC sẽ thực hiện chương trình phục vụ ngắt tại địa chỉ **0004H**.



2.1.3. Bộ nhớ Ngăn xếp (Stack Memory)

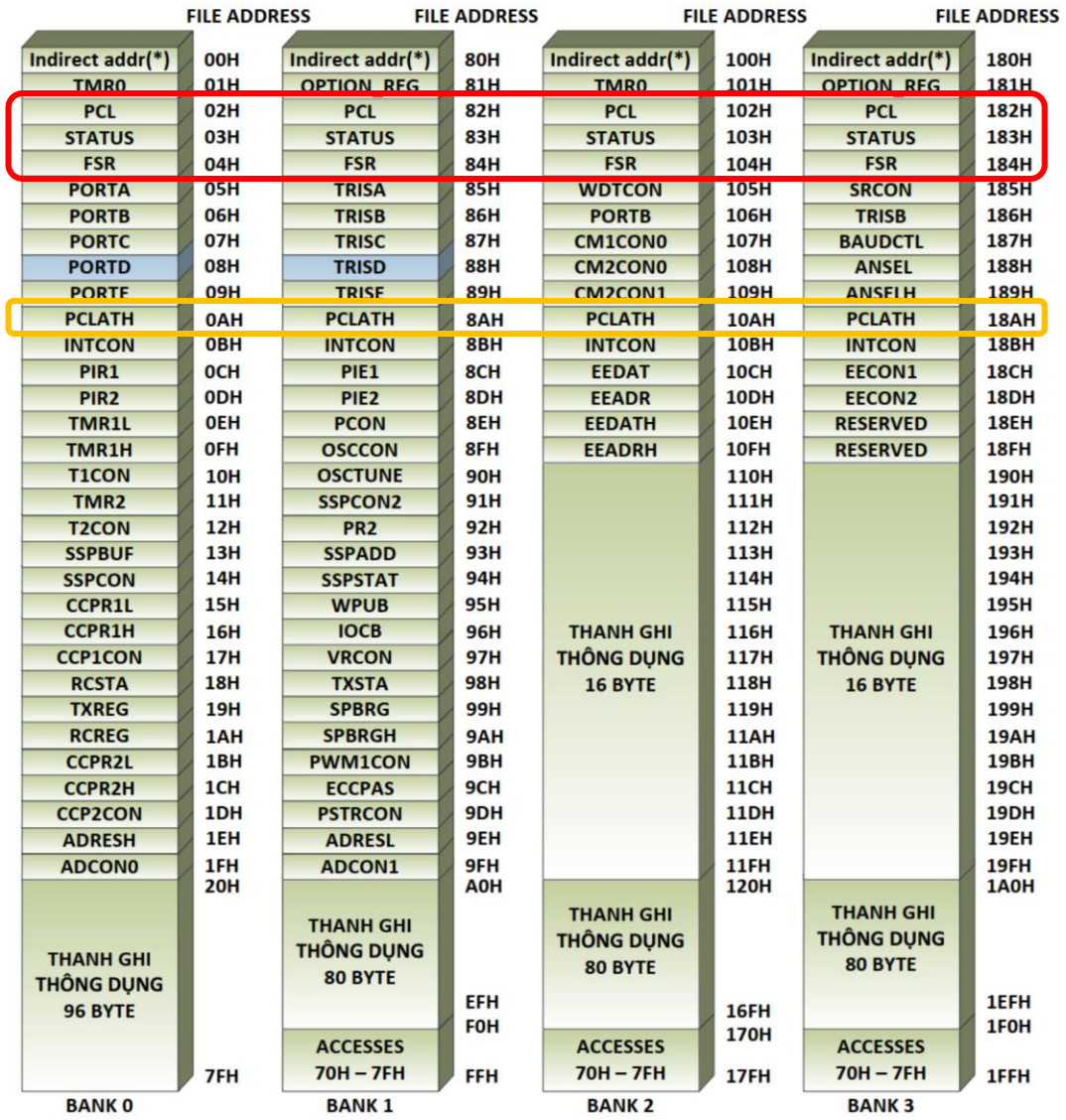
- **Tính độc lập:** Bộ nhớ ngăn xếp của PIC được thiết kế **độc lập** với bộ nhớ dữ liệu.
- **Chức năng:** Chỉ được sử dụng để lưu địa chỉ trở về (địa chỉ của thanh ghi PC) khi thực hiện lệnh gọi chương trình con hoặc khi xảy ra ngắt.
- **Dung lượng:** Chỉ có **8 ô nhớ** (stack level 1 đến 8). Do PC dài 13 bit, mỗi ô nhớ ngăn xếp cũng có chiều dài 13 bit. Điều này giới hạn các chương trình con lồng vào nhau tối đa là 8 cấp.



2.1.4. Bộ nhớ Dữ liệu (File Registers / RAM)

Toàn bộ các ô nhớ của bộ nhớ dữ liệu được gọi là File thanh ghi.

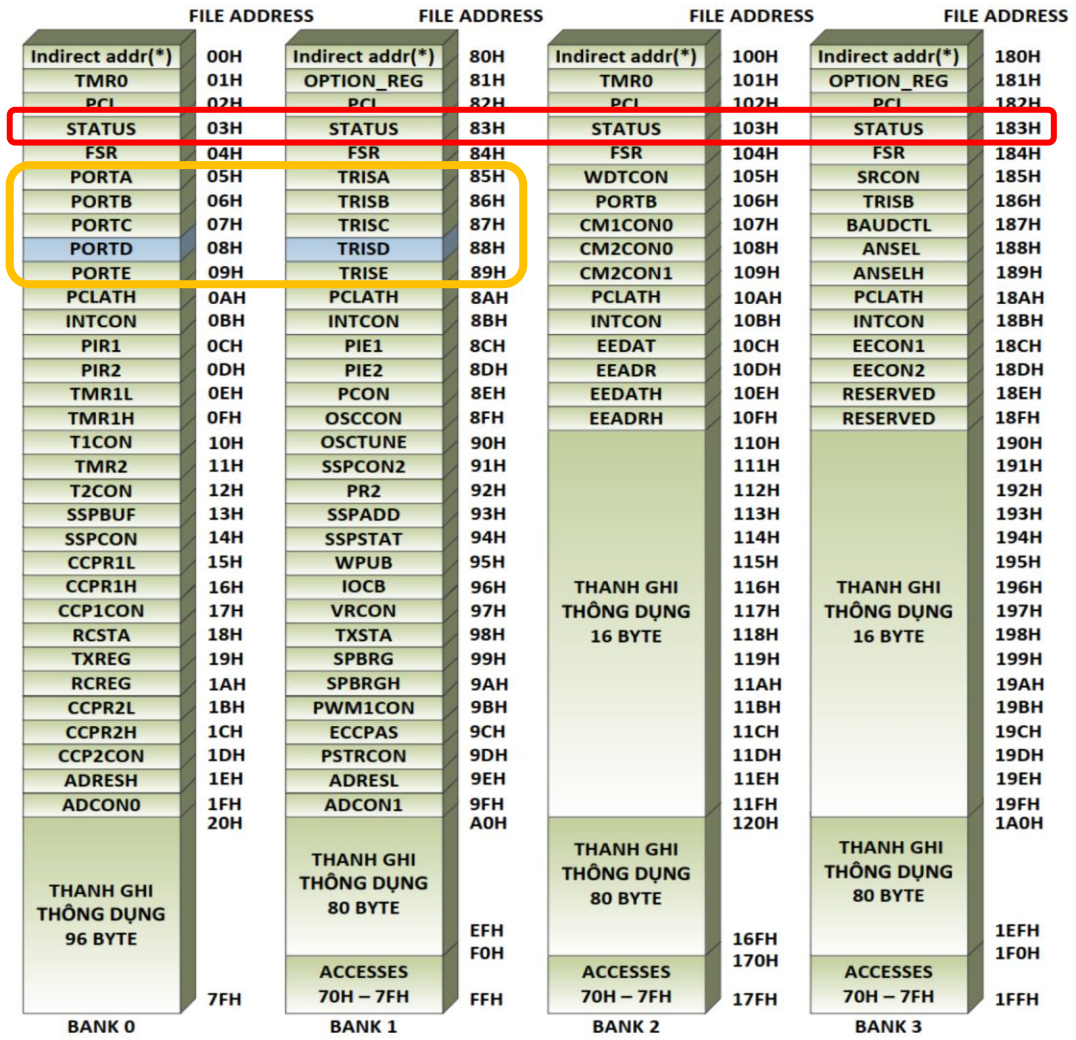
- **Dung lượng:** Tổng cộng 512 ô nhớ được chia thành **4 Bank**. Mỗi bank có dung lượng **128 byte**. Số lượng ô nhớ thực tế là 368.
- **Nội dung:** Bao gồm một số **thanh ghi chức năng đặc biệt** (SFR - Special Function Registers) nằm ở vùng địa chỉ thấp và các ô nhớ đa dụng (RAM-GPR (General Purpose Registers)). Đây chính là nơi lưu trữ các biến mà bạn khai báo trong code (ví dụ: int temp;).



• Cách truy xuất:

◦ **Truy xuất trực tiếp:** Các lệnh chỉ được phép truy xuất trong **1 bank** tại một thời điểm. Việc chuyển đổi bank được điều khiển bởi **hai bit RP1 và RP0** trong thanh ghi STATUS.

◦ **Truy xuất gián tiếp:** Cho phép truy xuất 2 bank (Bank 0, 1 hoặc Bank 2, 3). Việc chuyển đổi giữa hai nhóm bank này được điều khiển bởi **bit IRP** trong thanh ghi STATUS.



Ví dụ:

```
#include <xc.h>
void main(void) {
    // Không cần lệnh chuyển Bank thủ công
    ANSEL = 0x00;  // XC8 tự biết ANSEL nằm ở Bank 3
    TRISB = 0x00;  // XC8 tự biết TRISB nằm ở Bank 1
    PORTB = 0xFF;  // XC8 tự biết PORTB nằm ở Bank 0
    while(1);
}
```

```
BCF STATUS, RP1  ; RP1 = 0
BSF STATUS, RP0  ; RP0 = 1 -> Chọn Bank 1
MOVLW B'00000000'
MOVWF TRISB      ; Cấu hình PORTB là Output
BCF STATUS, RP0  ; Quay lại Bank 0 để điều khiển PORTB
```

2.1.5. Các thanh ghi có chức năng đặc biệt (SFR)

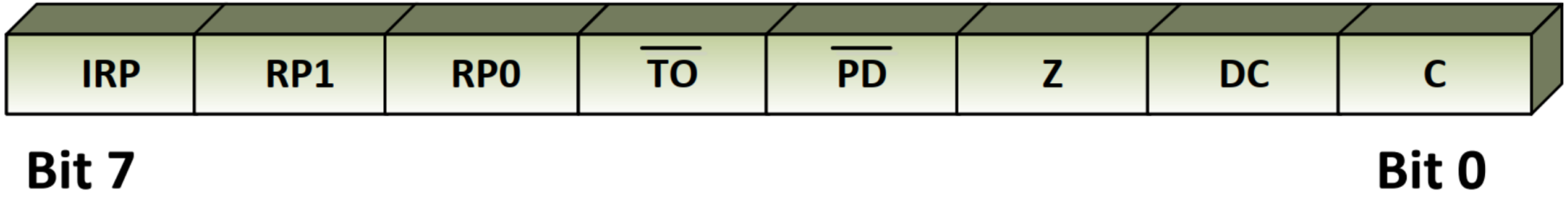
a. Các thanh ghi lõi (Core Registers)

- Đây là những thanh ghi xuất hiện ở tất cả các Bank để bạn có thể truy cập nhanh mà không cần chuyển Bank liên tục.
- **STATUS (03h, 83h, 103h, 183h):** Thanh ghi quan trọng nhất, chứa các bit trạng thái của bộ ALU (Zero, Carry) và đặc biệt là các bit chọn Bank (**RP0, RP1**).
- **INTCON (0Bh, 8Bh, 10Bh, 18Bh):** Điều khiển ngắt toàn cục và các ngắt cơ bản như ngắt Timer0, ngắt ngoài (RB0/INT).
- **PCL & PCLATH:** Điều khiển bộ đếm chương trình (Program Counter), dùng khi bạn thực hiện các lệnh nhảy hoặc gọi hàm.
- **FSR & INDF:** Dùng để truy cập bộ nhớ theo kiểu gián tiếp (con trỏ).

b. Thanh ghi Trạng thái (STATUS REGISTER)

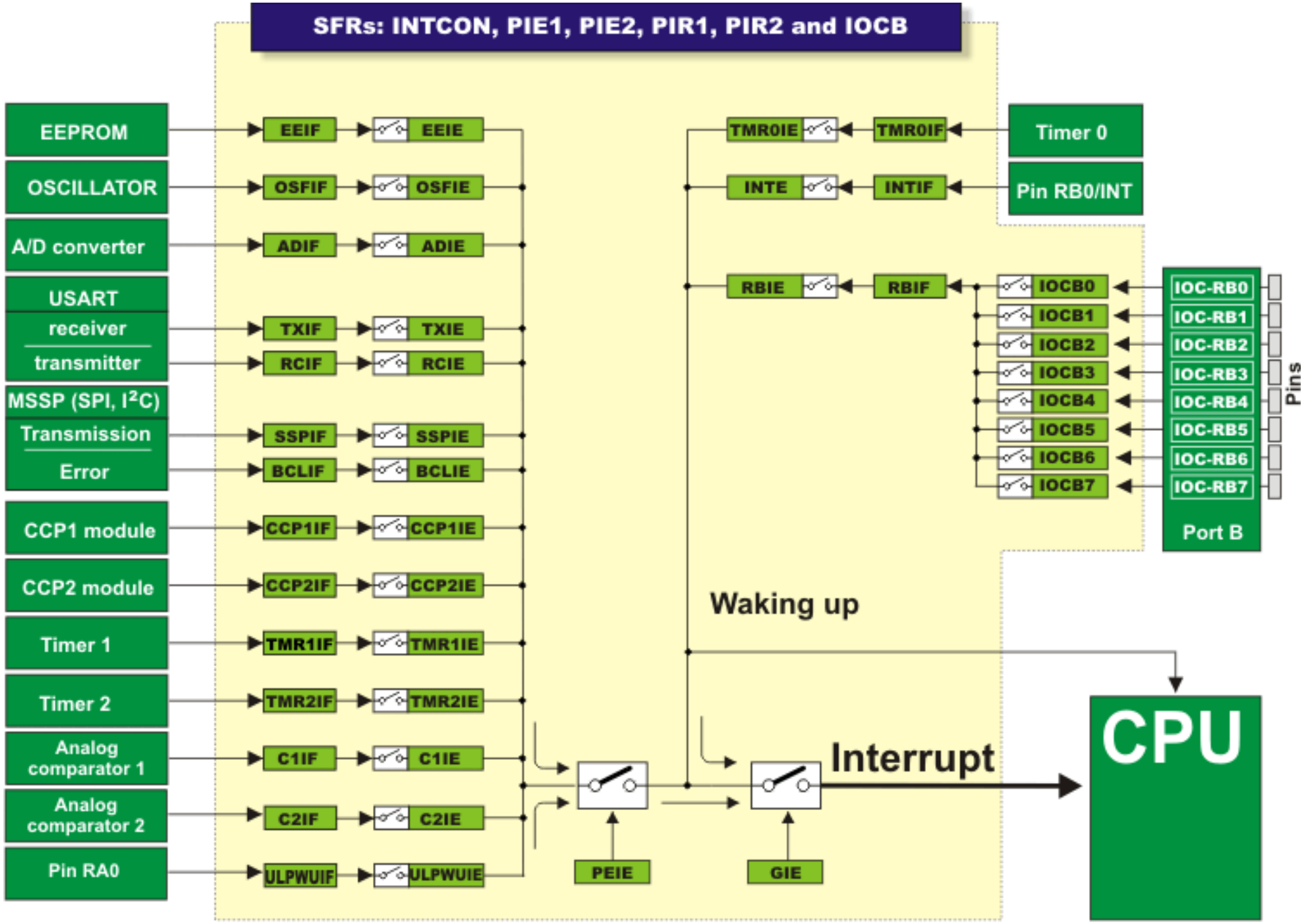
STATUS (03h, 83h, 103h, 183h):

- **Địa chỉ:** 03H, 83H, 103H, 183H (có mặt ở cả 4 bank).
- **Chức năng:** Chứa trạng thái của khối ALU, trạng thái Reset và các bit dùng để chọn bank bộ nhớ dữ liệu.



- **Các bit quan trọng:**
  - **IRP (Bit 7):** Bit lựa chọn thanh ghi cho truy xuất gián tiếp (chọn Bank 2, 3 hay Bank 0, 1).
  - **RP1:RP0 (Bit 6-5):** Các bit lựa chọn bank cho truy xuất trực tiếp (chọn Bank 0, 1, 2, hoặc 3).
  - **Z (Bit 2):** Cờ Zero. Bằng 1 nếu kết quả phép toán bằng 0.
  - **C (Bit 0):** Cờ Carry/Borrow. Bằng 1 khi kết quả phép toán có tràn.
  - **TO (Bit 4):** Time-out bit.
  - **PD (Bit 3):** Power-down bit.

c. Ngắt và thanh ghi điều khiển ngắt



Các thanh ghi ngắt

i. Thanh ghi INTCON (Interrupt CONTROL)

R/W(0)	R/W(0)	R/W(0)	R/W(0)	R/W(0)	R/W(0)	R/W(0)	R/W(X)
GIE	PEIE	T0IE	INTE	RBIE	T0IF	INTF	RBIF

Bit	Kí hiệu	Chức năng (cho phép = 1; cấm = 0)
INTCON.7	GIE	Bit cho phép/cấm toàn bộ các nguồn ngắt.
INTCON.6	PEIE	Bit cho phép ngắt ngoại vi
INTCON.5	T0IE	Bit cho phép ngắt timer T0.
INTCON.4	INTE	Bit cho phép ngắt ngoài.
INTCON.3	RBIE	Bit cho phép ngắt portB thay đổi.
INTCON.2	T0IF	Cờ báo ngắt của timer T0.
INTCON.1	INTF	Cờ báo ngắt của ngắt ngoài.
INTCON.0	RBIF	Cờ báo ngắt portB thay đổi.

ii. Thanh ghi PIE1 (Peripheral Interrupt Enable) và PIR1 (Peripheral Interrupt Request)

R/W(0)	R/W(0)	R/W(0)	R/W(0)	R/W(0)	R/W(0)	R/W(0)
-	ADIE	RCIE	TXIE	SSPIE	CCP1IE	TMR2IE	TMR1IE

R/W(0)	R/W(0)	R/W(0)	R/W(0)	R/W(0)	R/W(0)	R/W(0)
-	ADIF	RCIF	TXIF	SSPIF	CCP1IF	TMR2IF	TMR1IF

Bit	Kí hiệu	Chức năng (cho phép = 1; cấm = 0)
PIE1.7	-	Bit chưa có chức năng.
PIE1.6	ADIE	Bit cho phép ADC ngắt.
PIE1.5	RCIE	Bit cho phép ngắt nhận dữ liệu.
PIE1.4	TXIE	Bit cho phép ngắt phát dữ liệu.
PIE1.3	SSPIE	Bit cho phép ngắt truyền dữ liệu đồng bộ.
PIE1.2	CCP1IE	Bit cho phép ngắt khối CCP1.
PIE1.1	TMR2IE	Bit cho phép ngắt của timer T2.
PIE1.0	TMR1IE	Bit cho phép ngắt của timer T1.

Bit	Kí hiệu	Chức năng (ngắt xảy ra = 1; chưa xảy ra = 0)
PIR1.7	-	Bit chưa có chức năng.
PIR1.6	ADIF	Cờ báo ngắt của ADC.
PIR1.5	RCIF	Cờ báo ngắt nhận dữ liệu.
PIR1.4	TXIF	Cờ báo ngắt phát dữ liệu.
PIR1.3	SSPIF	Cờ báo ngắt truyền dữ liệu đồng bộ.
PIR1.2	CCP1IF	Cờ báo ngắt khối CCP1.
PIR1.1	TMR2IF	Cờ báo ngắt của timer T2.
PIR1.0	TMR1IF	Cờ báo ngắt của timer T1.

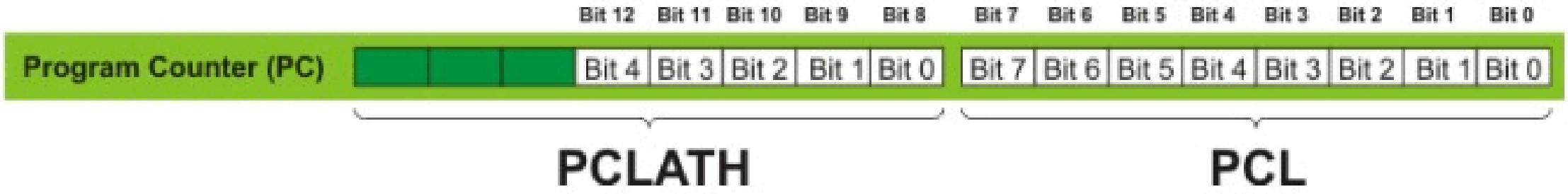
iii. Thanh ghi PIE2 (Peripheral Interrupt Enable) và PIR2 (Peripheral Interrupt Request)

R/W(0)	R/W(0)	R/W(0)	R/W(0)	R/W(0)	R/W(0)		R/W(0)
OSFIE	C2IE	C1IE	EEIE	BCLIE	ULPWUIE	-	CCP2IE

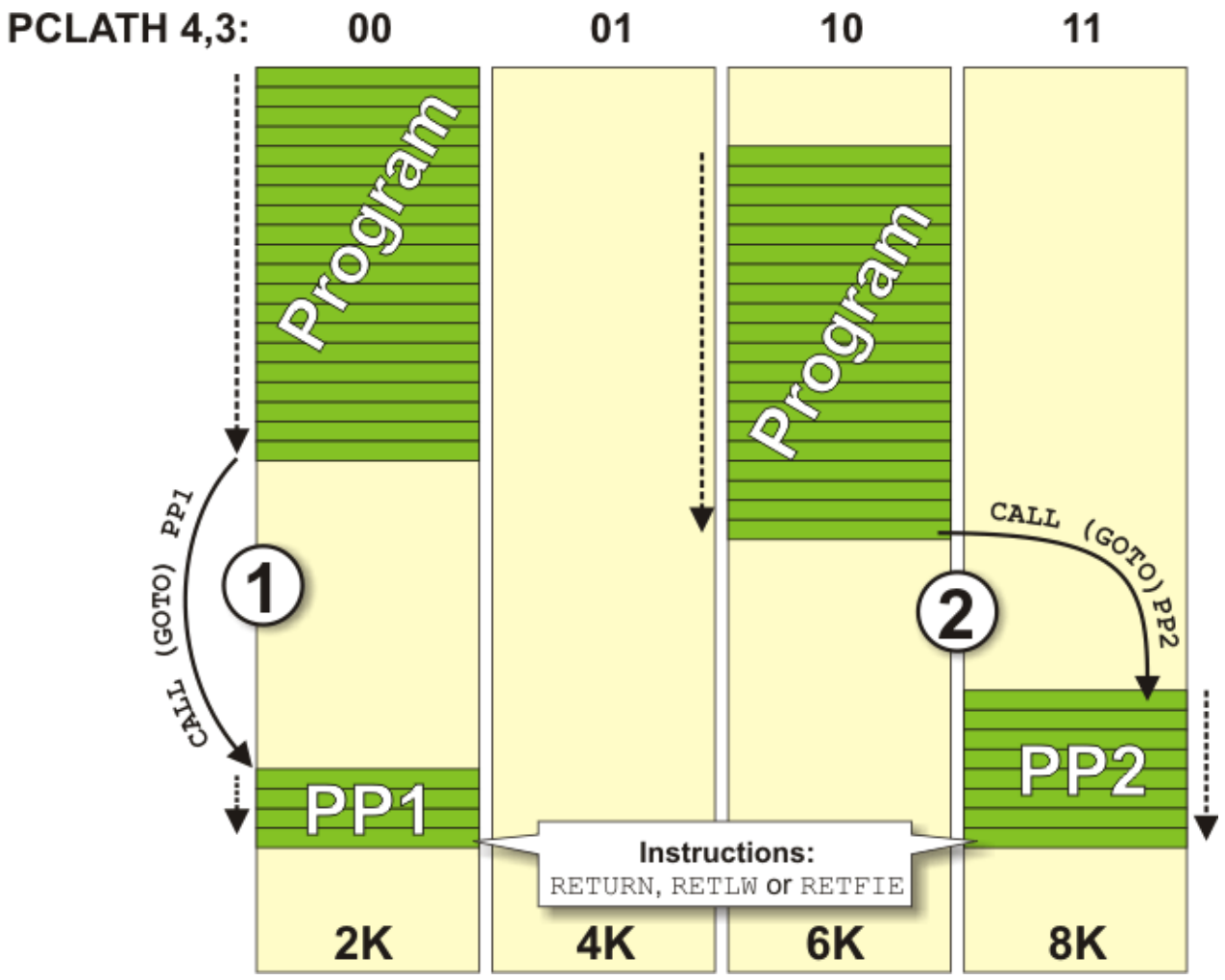
R/W(0)	R/W(0)	R/W(0)	R/W(0)	R/W(0)	R/W(0)		R/W(0)
OSFIF	C2IF	C1IF	EEIF	BCLIF	ULPWUIF	-	CCP2IF

Bit	Kí hiệu	Chức năng (cho phép = 1; cấm = 0)
PIE2.7	OSFIE	Bit cho phép ngắt của bộ dao động hồng.
PIE2.6	C2IE	Bit cho phép ngắt của khối so sánh 2.
PIE2.5	C1IE	Bit cho phép ngắt của khối so sánh 1.
PIE2.4	EEIE	Bit cho phép ngắt của bộ nhớ eeprom.
PIE2.3	BCLIE	Bit cho phép ngắt xung đột truyền dữ liệu I2C.
PIE2.2	ULWUIE	Bit cho phép ngắt của bộ đánh thức CPU với công suất cực thấp.
PIE2.1	-	Bit chưa có chức năng.
PIE2.0	CCP2IE	Bit cho phép ngắt của CCP2.

Bit	Kí hiệu	Chức năng (cho phép = 1; cấm = 0)
PIR2.7	OSFIF	Cờ báo ngắt của bộ dao động hồng.
PIR2.6	C2IF	Cờ báo ngắt của khối so sánh 2.
PIR2.5	C1IF	Cờ báo ngắt của khối so sánh 1.
PIR2.4	EEIF	Cờ báo ngắt của bộ nhớ eeprom.
PIR2.3	BCLIF	Cờ báo ngắt xung đột truyền dữ liệu I2C.
PIR2.2	ULWUIF	Cờ báo ngắt của bộ đánh thức CPU với công suất cực thấp.
PIR2.1	-	Bit chưa có chức năng.
PIR2.0	CCP2IF	Cờ báo ngắt của CCP2.



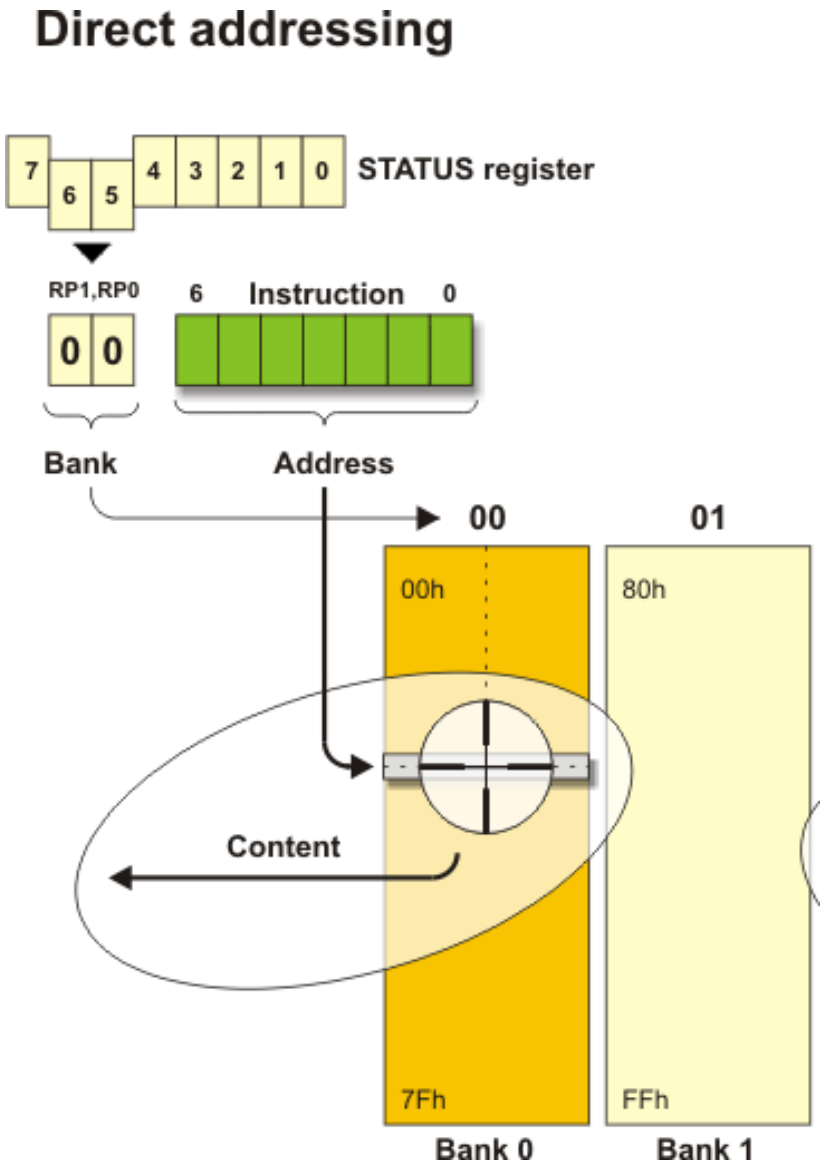
c. Thanh ghi điều khiển bộ đếm chương trình (Program counter (PC))



d. Truy cập bộ nhớ RAM trong PIC16F887

- Dữ liệu chỉ truy cập trong 1 bank.
- Để truy cập bank khác cần thay đổi địa chỉ bank tại RP0 và RP1

```
LIST    P=16F887
        INCLUDE "P16F887.INC"
        ORG     0x00
        GOTO    MAIN
;-----------------------------------------------
; Vùng dữ liệu (Bank0)
;-----------------------------------------------
        CBLOCK  0x20        ; Bank0
        VAR_A
        VAR_B
        ENDC
;-----------------------------------------------
; Chương trình chính
;-----------------------------------------------
MAIN:
        ;--- Chọn Bank0 trực tiếp ---
        BCF     STATUS, RP0 ; RP0=0
        BCF     STATUS, RP1 ; RP1=0 → Bank0
        ;--- Ghi trực tiếp vào VAR_A ---
        MOVLW   0x05        ; W = 0x05
        MOVWF   VAR_A       ; ghi trực tiếp vào VAR_A
        ;--- Đọc trực tiếp từ VAR_A và cộng ---
        MOVF    VAR_A, W    ; W = VAR_A (0x05)
        ADDLW   0x0B        ; W = W + 0x0B (5 + 11 = 16)
        ;--- Lưu kết quả vào VAR_B ---
        MOVWF   VAR_B
        ; Sau phép cộng, STATUS sẽ tự động cập nhật:
        ; Z = 0 (kết quả ≠ 0)
        ; DC = 0 (không tràn từ bit 3 sang bit 4)
        ; C = 0 (không tràn toàn phần)
        END
```

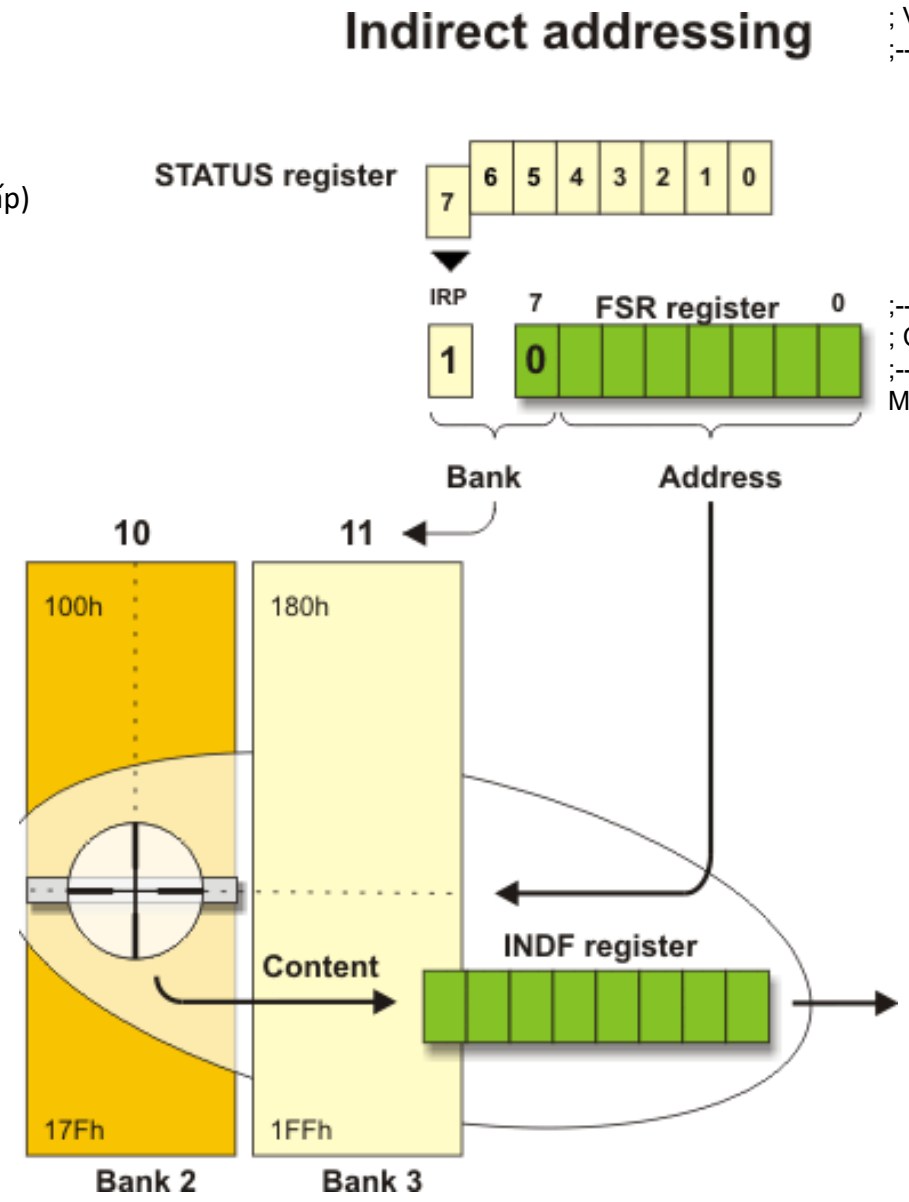



```
;-----------------------------------------------
; Vùng dữ liệu
;-----------------------------------------------
        CBLOCK  0x20        ; Bank0
        VAR0
        ENDC

        CBLOCK  0xA0        ; Bank2
        VAR2
        ENDC
;-----------------------------------------------
; Chương trình chính
;-----------------------------------------------
MAIN:
        ;--- Chọn Bank0 ---
        BCF     STATUS, RP0 ; RP0=0
        BCF     STATUS, RP1 ; RP1=0 → Bank0
        ;--- Ghi trực tiếp vào VAR0 ---
        MOVLW   0x05
        MOVWF   VAR0        ; ghi trực tiếp vào Bank0
        ;--- Chọn Bank2 ---
        BCF     STATUS, RP0 ; RP0=0
        BSF     STATUS, RP1 ; RP1=1 → Bank2
        ;--- Ghi trực tiếp vào VAR2 ---
        MOVLW   0x0A
        MOVWF   VAR2        ; ghi trực tiếp vào Bank2
        ;--- Đọc lại VAR2 và thực hiện phép cộng ---
        MOVF    VAR2, W     ; W = nội dung VAR2 (0x0A)
        ADDLW   0xF6        ; W = W + 0xF6
        ;--- Quay lại Bank0 để lưu kết quả ---
        BCF     STATUS, RP0
        BCF     STATUS, RP1 ; RP1=0, RP0=0 → Bank0
        MOVWF   VAR0        ; ghi kết quả vào VAR0
        END
```

d. Dùng để truy cập bộ nhớ theo kiểu gián tiếp **FSR & INDF:**

```
        LIST    P=16F887
            INCLUDE "P16F887.INC"
            ORG     0x00
            GOTO    MAIN
;-----------------------------------------------------
; Vùng dữ liệu
;-----------------------------------------------------
    CBLOCK  0x20        ; Bank0
    VAR0
    ENDC
    CBLOCK  0x60        ; Bank1 (địa chỉ > 0x50, vẫn trong vùng thấp)
    VAR1
    ENDC
;-----------------------------------------------------
; Chương trình chính
;-----------------------------------------------------
MAIN:
    ;--- Ghi gián tiếp vào VAR0 (Bank0) ---
    BCF     STATUS, IRP ; IRP=0 → vùng RAM thấp (Bank0–Bank1)
    MOVLW   LOW VAR0
    MOVWF   FSR
    MOVLW   0x05
    MOVWF   INDF        ; ghi gián tiếp vào VAR0
    ;--- Ghi gián tiếp vào VAR1 (Bank1) ---
    ; vẫn IRP=0 vì Bank1 nằm trong vùng thấp
    MOVLW   LOW VAR1
    MOVWF   FSR
    MOVLW   0x0A
    MOVWF   INDF        ; ghi gián tiếp vào VAR1
    ;--- Đọc lại VAR1 bằng gián tiếp ---
    MOVF    INDF, W     ; W = nội dung VAR1 (0x0A)
    ADDLW   0xF6        ; thực hiện phép cộng để quan sát STATUS
    ;--- Lưu kết quả vào VAR0 ---
    MOVLW   LOW VAR0
    MOVWF   FSR
    MOVWF   INDF        ; ghi kết quả vào VAR0
    END
```

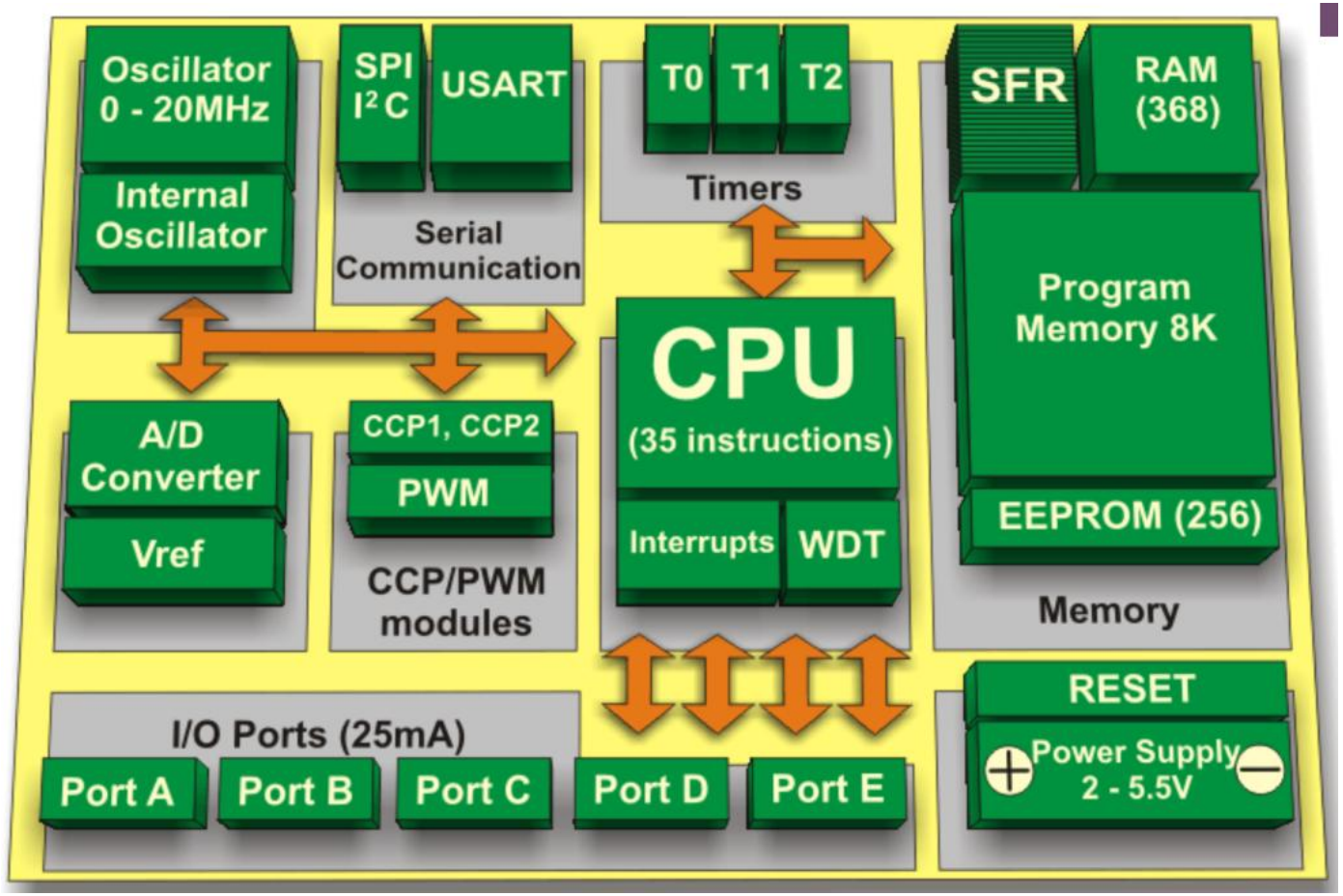



```
;-----------------------------------------------------
; Vùng dữ liệu
;-----------------------------------------------------
    CBLOCK  0x20        ; Bank0
    VAR1
    ENDC

    CBLOCK  0xA0        ; Bank2 (địa chỉ cao hơn 0x7F)
    VAR2
    ENDC
;-----------------------------------------------------
; Chương trình chính
;-----------------------------------------------------
MAIN:
    ;--- Ghi gián tiếp vào VAR1 (Bank0) ---
    MOVLW   LOW VAR1    ; địa chỉ thấp của VAR1
    MOVWF   FSR         ; FSR = địa chỉ VAR1
    MOVLW   0x05
    MOVWF   INDF        ; ghi gián tiếp vào VAR1
    ;--- Ghi gián tiếp vào VAR2 (Bank2) ---
    BSF     STATUS, IRP ; bật IRP để chọn vùng RAM cao
    MOVLW   LOW VAR2    ; địa chỉ thấp của VAR2
    MOVWF   FSR
    MOVLW   0x0A
    MOVWF   INDF        ; ghi gián tiếp vào VAR2

    ;--- Đọc lại VAR2 bằng gián tiếp ---
    MOVF    INDF, W     ; W = nội dung VAR2
    ADDLW   0xF6
                        ; thực hiện phép cộng để quan sát STATUS
    ;--- Lưu kết quả vào VAR1 ---
    BCF     STATUS, IRP ; quay lại vùng RAM thấp
    MOVLW   LOW VAR1
    MOVWF   FSR
    MOVWF   INDF        ; ghi kết quả vào VAR1
    END
```

2.1.6. Bộ nhớ Dữ liệu EEPROM



- Dù không nằm trong RAM hay Flash chính, PIC16F887 còn có **256 Bytes EEPROM**.
  - **Dung lượng: 256 byte**.
  - **Chức năng:** Dùng để lưu trữ dữ liệu quan trọng, dữ liệu này vẫn còn nguyên vẹn khi mất điện. Bộ nhớ này cho phép xóa và lập trình 1,000,000 lần và có thể tồn tại trên 40 năm.